Propuesta de Arquitectura – ReUseITESO

Tipo de arquitectura propuesta

Monolito modular (backend) + frontend desacoplado

Descripción breve

El sistema se propone como una aplicación web desarrollada con un backend en Django utilizando una arquitectura monolítica modular, y un frontend en Next.js que consume una API REST. Esta aproximación permite mantener la simplicidad necesaria para un proyecto académico de un semestre, sin perder organización ni claridad en la separación de responsabilidades.

División del sistema en módulos

Backend (Django – API REST)

1. Módulo Core
   Se encarga de la base del sistema y es utilizado por todos los demás módulos.
   Incluye la autenticación y autorización de usuarios mediante correo institucional ITESO, la gestión de roles (vendedor, comprador y administrador), así como la configuración general y servicios comunes del sistema.
2. Módulo Marketplace
   Responsable de la funcionalidad principal de la aplicación.
   Permite la publicación de productos de segunda mano, su edición y eliminación, así como la consulta de publicaciones. Maneja la categorización de los productos y su estado dentro del sistema.
   Estados posibles de una publicación: publicado, reservado, vendido, cancelado.

Endpoints principales expuestos por este módulo incluyen la consulta de publicaciones, la creación de nuevas publicaciones, el acceso al detalle de un producto y el cambio de estado de una publicación.

3. Módulo Transacciones
   Gestiona el flujo de compra y venta entre usuarios.
   Administra la creación de transacciones, el historial de movimientos y el cambio de estados durante el proceso.
   Estados posibles de una transacción: pendiente, confirmada, completada, cancelada.
   Este módulo depende de la información del Marketplace para validar

disponibilidad de productos y del Core para identificar a los usuarios involucrados.

4. Módulo Comunicación
   Permite la comunicación directa entre compradores y vendedores.
   Incluye el envío de mensajes asociados a una publicación o transacción, así como notificaciones básicas dentro del sistema o por correo electrónico.
5. Módulo IA (Transversal)
   Funciona como un módulo de apoyo que consume información de otros módulos.
   Proporciona sugerencias de precios para publicaciones, detección básica de posibles publicaciones irregulares y recomendaciones simples de productos.
   Su implementación se plantea de forma progresiva a partir del MVP.

Frontend (Next.js)

Se encarga de la interfaz de usuario y la experiencia visual.

Incluye vistas como inicio de sesión, listado de productos, detalle de publicación y panel de usuario.

Gestiona formularios de publicación y consumo de la API REST, sin acceso directo a la lógica de negocio.

Comunicación entre módulos

La comunicación entre los módulos del backend se realiza de forma interna dentro del monolito, mediante servicios bien definidos.

La comunicación externa se realiza exclusivamente a través de la API REST en formato JSON.

El frontend interactúa únicamente con la API, sin acceso directo a la lógica interna del backend.

Dependencias con otros equipos

El frontend depende del backend a través de contratos de API claramente definidos.

El módulo de IA consume datos del Marketplace y de Transacciones.

El módulo Core es transversal y requerido por todos los demás módulos.

Información que consume cada módulo

El módulo Core consume credenciales, roles y configuraciones generales.

El módulo Marketplace consume información de usuarios y categorías.

El módulo de Transacciones consume información de productos y usuarios.

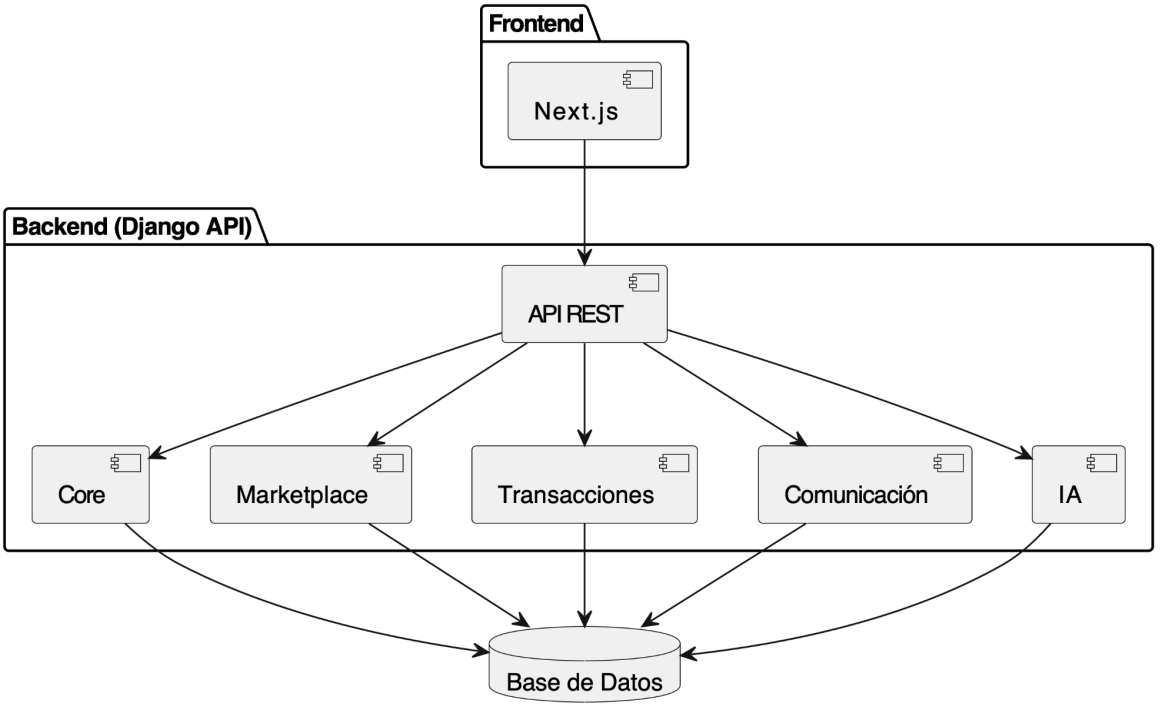
El módulo de Comunicación consume información de usuarios y transacciones.

El módulo de IA consume datos de publicaciones y el historial de ventas.

Información que expone

El sistema expone una API REST organizada por módulos.

Las respuestas se entregan en formato JSON estandarizado.

La autenticación se maneja mediante tokens.

Diagrama